

اليوم 61 - REST APIs

API أو Application Programming Interface هو مفهوم يسمح لتطبيقات برمجيين بالتواصل مع بعضهما البعض بشكل منظم وموحد. فكر في API كوسيط أو نادل في مطعم: أنت (التطبيق) تطلب طعاما معيناً (بيانات أو خدمة) من المطبخ (النظام الخلفي أو الجهاز)، والنادل (API) ينقل طلبك ويعود بالطعام المطلوب. بدون API، لا يمكن للتطبيقات المختلفة أن تتفاعل مع بعضها بسهولة. في سياق الشبكات، توفر أجهزة سيسكو وأنظمة مثل Cisco DNA Center و Cisco Catalyst Center و Cisco Meraki واجهات API تسمح للمطورين ومهندسي الشبكات بالتفاعل مع هذه الأنظمة برمجياً، بدلاً من استخدام واجهة المستخدم الرسومية (GUI) أو CLI التقليدية. هذا هو أساس أتمتة الشبكات الحديثة.

REST أو Representational State Transfer هو أسلوب معماري (Architectural Style) لبناء APIs يعتمد على مبادئ بسيطة: كل شيء هو مورد (Resource) يمكن الوصول إليه عبر URL فريد (Uniform Resource Identifier أو URI)، ويتم التعامل مع هذه الموارد باستخدام طرق HTTP القياسية. REST API ليس معياراً في حد ذاته، بل مجموعة من المبادئ التوجيهية التي تجعل APIs بسيطة وسهلة الاستخدام. أشهر مبادئ REST هي: عديمة الحالة (Stateless) أي أن كل طلب مستقل ولا يعتمد على الطلبات السابقة، كل مورد له URI محدد، استخدام طرق HTTP بشكل صحيح، إرجاع البيانات بصيغة منظمة (غالباً JSON). REST APIs أصبحت المعيار الفعلي لبناء APIs في جميع المجالات، بما في ذلك إدارة الشبكات، وذلك بسبب بساطتها واعتمادها على HTTP الذي يفهمه الجميع.

طرق (HTTP Methods) هي الإجراءات التي يمكنك تنفيذها على الموارد في REST API. الطريقة الأولى والأكثر استخداماً هي GET، وتستخدم لقراءة أو استرجاع البيانات من الخادم. على سبيل المثال، طلب GET إلى API لسرد جميع الأجهزة في الشبكة. الطريقة الثانية هي POST، وتستخدم لإنشاء مورد جديد. مثال: إضافة جهاز جديد إلى نظام إدارة الشبكة. الطريقة الثالثة هي PUT، وتستخدم لاستبدال مورد بالكامل. مثال: تحديث التكوين الكامل لجهاز معينة. الطريقة الرابعة هي PATCH، وتستخدم لتحديث جزء من مورد (تعديل جزئي). مثال: تغيير اسم الجهاز فقط. الطريقة الخامسة هي DELETE، وتستخدم لحذف مورد. مثال: حذف جهاز من نظام الإدارة. كل طريقة لها معنى محدد ويجب استخدامها بالشكل الصحيح لبناء API متسق ومفهوم. طلب GET يجب ألا يغير أي شيء على الخادم، بينما طلب POST يمكن أن ينشئ موارد جديدة.

رموز حالة (HTTP Status Codes) هي جزء أساسي من أي REST API. عندما ترسل طلباً، يعود الخادم برمز حالة (Status Code) يخبرك بنتيجة الطلب. رموز 2xx تعني النجاح: 200 OK (نجاح عام)، 201 Created (تم إنشاء المورد بنجاح)، 204 No Content (نجاح لكن لا يوجد محتوى في الرد). رموز 4xx تعني خطأ في الطلب من العميل: 400 Bad Request (طلب غير صحيح)، 401 Unauthorized (غير مصرح، تحتاج مصادقة)، 403 Forbidden (ممنوع حتى مع المصادقة)، 404 Not Found (المورد غير موجود)، 422 Validation Error (بيانات الطلب غير صالحة). رموز 5xx تعني خطأ في الخادم: 500 Internal Server Error (خطأ عام في الخادم)، 503 Service Unavailable (الخدمة غير متاحة مؤقتاً). فهم هذه الرموز أساسي لاستكشاف أخطاء API وإصلاحها. إذا حصلت على 401، فالمشكلة في المصادقة. إذا حصلت على 404، فالمسار (URL) غير صحيح. إذا حصلت على 500، فالمشكلة من الخادم وليس منك.

طرق HTTP ورموز الحالة الشائعة:

GET: قراءة بيانات (200 OK). إنشاء مورد جديد (201 PUT). (Created): استبدال مورد (200 OK). PATCH: تحديث جزئي (200 DELETE). (OK): حذف مورد (204 No Content). رموز الخطأ: 401 غير مصرح، 403 ممنوع، 404 غير موجود، 500 خطأ خادم.

مصادقة (API Authentication) هي عملية التحقق من هوية الطالب (العميل) قبل السماح له باستخدام API. هناك عدة طرق للمصادقة في REST APIs. الطريقة الأولى هي API Key: حيث يحصل العميل على مفتاح فريد (سلسلة نصية طويلة) من النظام، ويرسل هذا المفتاح في رأس (Header) الطلب في كل مرة. هذه طريقة بسيطة لكنها ليست آمنة جداً. الطريقة الثانية هي Basic Authentication: حيث يتم إرسال اسم المستخدم وكلمة المرور مشفرين بـ Base64 في رأس Authorization. هذا بسيط لكن Base64 ليس تشفيراً حقيقياً (يمكن فك تشفيره بسهولة)، لذلك يجب استخدام HTTPS دائماً معه. الطريقة الثالثة هي Bearer Token: حيث يسجل العميل دخوله أولاً ويحصل على رمز مميز (Token) صالح لفترة زمنية محددة (مثلاً ساعة)، ثم يرسل هذا الـ Token في كل طلب لاحق. JWT (JSON Web Token) هو الشكل الأكثر استخداماً للـ Tokens. الطريقة الرابعة والأكثر تقدماً هي OAuth 2.0، وهو بروتوكول مصادقة معقد يتيح للعميل الحصول على صلاحيات محددة (Scopes) لتطبيقات الطرف الثالث دون مشاركة كلمة المرور. Cisco DNA Center يستخدم Bearer Token للمصادقة: تسجل دخولك باستخدام POST مع اسم المستخدم وكلمة المرور، وتستلم Token، ثم تستخدمه في كل طلب لاحق.

لنطبق ما تعلمناه على مثال عملي مع Cisco DNA Center REST API. أولاً، نحتاج إلى معرفة عنوان URL الأساسي للـ API (Base URL)، مثلاً <https://dna-center-server/api>. لإحضار قائمة الأجهزة، نستخدم طلب GET إلى <https://dna-center-server/api/dna/intent/api/v1/network-device/>. قبل ذلك، يجب أن نحصل على Token مصادقة. نرسل طلب POST إلى <https://dna-center-server/api/dna/system/api/v1/auth/token/> مع اسم المستخدم وكلمة المرور في Basic Auth. يستجيب الخادم بـ Token نضعه في رأس Authorization كـ Bearer Token في جميع الطلبات اللاحقة. لنرى كيف يبدو هذا في بايثون باستخدام مكتبة requests.

مثال على استخدام REST API لـ Cisco DNA Center في بايثون:

```

import requests
import json

BASE_URL = "https://dna-center-server/api"

# الحصول على Token
auth_response = requests.post(
    f"{BASE_URL}/dna/system/api/v1/auth/token",
    auth=("username", "password"),
    verify=False
)
token = auth_response.json()["Token"]

# جلب الأجهزة باستخدام Token
headers = {"X-Auth-Token": token}
devices_response = requests.get(
    f"{BASE_URL}/dna/intent/api/v1/network-device",
    headers=headers,
    verify=False
)

if devices_response.status_code == 200:
    devices = devices_response.json()["response"]
    for device in devices:
        print(f"Hostname: {device['hostname']}, IP: {device['managementIpAddress']}")
else:
    print(f"Error: {devices_response.status_code}")

```

تنسيق البيانات في استجابة API هو JSON عادة. في مثال DNA Center أعلاه، الاستجابة تحتوي على حقل response وهو مصفوفة من الأجهزة. كل جهاز يحتوي على حقول مثل hostname و managementIpAddress و platformId و series و softwareType و collectionStatus وغيرها. يمكنك استخدام هذه البيانات لبناء لوحات معلومات (Dashboards) مخصصة، أو لأتمتة مهام الصيانة والدعم. مثلاً، يمكنك كتابة سكريبت يفحص جميع الأجهزة في الشبكة بشكل دوري ويكتب تقريراً عن حالة كل جهاز. أو سكريبت يقوم باكتشاف الأجهزة الجديدة وإضافتها تلقائياً إلى نظام المراقبة. أو سكريبت يسحب تكوينات الأجهزة احتياطياً بشكل دوري (Backup Configuration). الاحتمالات لا حصر لها.

في الختام، REST APIs هي جوهر أتمتة الشبكات الحديثة. كل منصة إدارة شبكات رئيسية توفر REST API، وفهم كيفية استخدامها هو مهارة أساسية لمهندس الشبكات العصري. يجب أن تتعلم كيفية قراءة توثيق API (API Documentation)، وكيفية اختبار APIs باستخدام أدوات مثل Postman أو curl، وكيفية دمج APIs في سكريبتات بايثون. تذكر أن الممارسات الجيدة تشمل: التعامل مع أخطاء الشبكة (Network Errors) بشكل مناسب، إعادة المحاولة عند فشل الطلب (Retry Logic)، احترام حدود معدل الطلبات (Rate Limits)، وتأمين بيانات المصادقة (لا تضعها في النص البرمجي مباشرة بل استخدم متغيرات بيئية). API ليس مجرد

واجهه تقنيه، بل هو بوابة لأتمتة كل شيء في الشبكة.